

LABORATORIO DI INGEGNERIA DEI SISTEMI SOFTWARE

Introduction

Questo documento descrive il lavoro svolto nello **Sprint 0** per lo sviluppo del sistema **cargoservice**. L'obiettivo principale dello sprint 0 è analizzare, formalizzare e rendere non ambiguo il linguaggio utilizzato per descrivere i requisiti, chiarendo le entità in gioco. L'automatizzazione effettiva delle operazioni di carico dei container nella stiva di una nave rappresenta invece il **core business** del sistema finale, la cui architettura verrà delineata progressivamente.

Requirements

Il testo completo dei requisiti fornito dal committente è disponibile al seguente link: <Tema2026.html> (../Tema2026.html)

The company asks us to build service named **cargoservice** that should work as follows.

The **cargoservice** is able to receive a **request to load** a container sent by some customer by using the **pushbutton** of the **IOPort**.

- It sends the answer **retrylater**, if the **IOPort** is currently occupied by a container or if the system is **Out of service**
- It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the **request to load** is accepted, the customer must move the container in the **sensor** area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the **cargoservice** uses the **cargorobot** to move the container from the **IOPort** to the **slot5** (for marking the container) and then to the reserved slot.

The service must also show on the **display** on the **IOPort**:

- the current state of the **hold**
- the message '**Service working**', when it is all is going well
- the message '**Out of service**' if the **sonar sensor** measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the **sonar**).

Requirement analysis | Vocabolario

Al fine di abbattere le ambiguità, definiamo un dizionario in formato discorsivo in cui esploriamo prima il significato funzionale e di dominio dei termini, per poi mapparli (dove opportuno) sulle logiche computazionali ad attori.

Nota Metodologica: Il Metamodello QAK ([Descrizione QAK](https://anatali.github.io/issLab2026/_static/docs/Protobook.pdf#chapter.17))

(https://anatali.github.io/issLab2026/_static/docs/Protobook.pdf#chapter.17)

Per la modellazione formale e l'implementazione adotteremo il linguaggio **QAK**.

Un **QActor** è un'entità computazionale autonoma, reattiva e message-driven, nativamente predisposta per operare in sistemi distribuiti. Da qui in avanti, quando identificheremo un componente attivo del dominio, lo designeremo esplicitamente come **QActor**, assoggettandolo alla rigorosa semantica di tale linguaggio.

IOPort (WebGUI)

Nei requisiti si parla di un **pushbutton** e di un **display** sull'IOPort. Essendo una scelta progettuale approvata dal committente, questi due elementi non sono trattati come dispositivi fisici separati, bensì costituiscono un'unica interfaccia utente software, ovvero una **WebGUI**. Tale interfaccia sarà ospitata sul medesimo nodo hardware deputato all'esecuzione del servizio principale. Di conseguenza, l'**IOPort** rappresenta il punto d'accesso software per l'utente, un'interfaccia esterna capace di dialogare tramite i comuni protocolli di rete (es. HTTP o WebSocket).

VirtualRobot e Cargorobot

Il sistema deve pilotare un robot fisico o simulato. Considerando i molteplici ambienti e robot virtuali pre-forniti a disposizione per la simulazione e il collaudo (VirtualRobot26, RobotObj26, RobotService26, RobotSmart26), l'analisi ha decretato l'adozione di **RobotSmart26** come la scelta architetaturalmente ottimale per questo progetto. Questa decisione scaturisce dal fatto che RobotSmart26 non si limita ad eseguire comandi cinematici di base, ma integra nativamente funzionalità di movimento autonomo e pianificazione dei percorsi, sollevando l'orchestratore dal gravoso onere del calcolo millimetrico delle traiettorie.

È fondamentale chiarire che il robot virtuale (in questo caso RobotSmart26) è un'**entità esterna e separata** rispetto al nostro software. Il **cargorobot**, invece, è il **QActor logico interno** che noi svilupperemo. Esso funge da "driver" intelligente: riceve comandi logici

di alto livello (es. "vai allo slot 1") e li traduce in istruzioni comprensibili dal **RobotSmart26** sottostante.

Cargoservice, Stiva (hold), Sonar e Led

Il **cargoservice** è l'orchestratore dell'intero sistema. Implementa la logica di business pura, valuta le condizioni della stiva (piena/libera) e del sensore, e dirige il **cargorobot**. Computazionalmente, è un **QActor** reattivo centrale.

La **stiva (hold)** racchiude le postazioni fisiche destinate all'immagazzinamento dei container (`slots1-4`) e alla marcatura temporanea (`slot5`). Essa non costituisce un attore autonomo (poiché non ha comportamento attivo autonomo), bensì viene formalmente modellata all'interno dello stato dell'orchestratore `cargoservice` (QActor) mediante variabili di stato booleane. Ciascuno slot logico viene mappato in coordinate cartesiane precise sulla griglia bidimensionale del simulatore

`RobotSmart26` (visualizzata a fianco del robot virtuale):

- `home` → coordinate `(0,0)`
- `ioport` → coordinate `(4,0)`
- `slot5` → coordinate `(4,3)`
- `slot1` → coordinate `(1,1)`
- `slot2` → coordinate `(1,4)`
- `slot3` → coordinate `(3,2)`
- `slot4` → coordinate `(3,4)`

Le variabili di stato per tracciare l'occupazione all'interno dell'attore `cargoservice` sono:

```
var Slot1Occupied = false
var Slot2Occupied = false
var Slot3Occupied = false
var Slot4Occupied = false
```

Il **sonar** è un dispositivo di rilevazione della distanza. Dai requisiti si evince che il sistema deve passare in uno stato di **Out of service** se il sonar misura una distanza `D > DFREE` per almeno 3 secondi (indice di una possibile rottura del sensore). Il sonar viene modellato formalmente come un **QActor** indipendente (esecutore in un thread separato) che emette periodicamente messaggi di tipo dispatch (`distance(D)`) per permettere all'orchestratore `cargoservice` di valutare la condizione temporale e aggiornare lo stato del sistema.

Il **led** è un dispositivo luminoso. Pur essendo passivo per natura, poiché si suppone distribuito su una macchina hardware a sé stante insieme al sonar (es. un Raspberry Pi), è opportuno modellarlo come **QActor** in ascolto di messaggi di stato per attivare o disattivare il lampeggio.

Modello delle Interazioni Logiche (Base) e Modello Eseguitibile

Tra le interazioni descritte nei requisiti, individuiamo fin da subito il pattern comunicativo cardine per l'utente:

- **Request / Reply** (Interazione sincrona o asincrona che attende risposta): Utilizzata dalla WebGUI (IOPort) verso il `cargoservice` per la richiesta di scarico (`request load : load(X)`). A tale richiesta seguirà necessariamente una `reply` contenente l'assegnazione dello slot o un messaggio di rifiuto (`retrylater`).

Il risultato concreto di questa fase di analisi è la definizione di un primo **modello eseguibile dell'architettura desunta dai requisiti**. Il file QAK preliminare (disponibile in `cargoservice.qak` (<https://github.com/riccardocar99/cargoservice-/blob/main/sprint0/src/cargoservice.qak>)) modellerà esclusivamente i componenti qui descritti e i loro scambi di messaggi base, offrendo una rappresentazione logicamente validabile prima ancora di scendere nei dettagli architetturali.

Problem analysis

Verso un Modello dei Requisiti

L'infrastruttura QAK ci permette di affrontare le problematiche relative ai sistemi distribuiti. Di seguito esploriamo i nodi fondamentali dell'architettura.

Abstraction Gap

Esiste un chiaro **abstraction gap** tra le operazioni di business richieste dall'orchestratore (es. "sposta il robot allo slot 2") e i comandi cinematico-cartesiani di basso livello previsti da `RobotSmart26`. Il problema viene risolto introducendo il QActor **cargorobot**: esso espone all'orchestratore `cargoservice` un contratto di messaggistica formale Request-Reply (richiesta `move_robot(DEST)` e risposta `move_done(RESULT)`) e si occupa internamente di effettuare la traduzione logico-cartesiana e la gestione della comunicazione di rete TCP/CoAP verso il robot virtuale, colmando così il divario astrattivo.

Definizione dei Contesti (Nodi di Deployment)

Poiché abbiamo più componenti software che girano su macchine e ambiti differenti, l'architettura logica richiede la definizione di almeno tre **contesti** QAK separati:

- `ctx_cargoservice`: Il contesto principale hostato sul PC. Questo nodo conterrà le entità decisionali e orchestranti, in particolare l'attore **cargoservice** e l'attore **cargorobot**. Inoltre, su questa stessa macchina hardware verrà esposta la WebGUI.
- `ctx_gui`: Il contesto logico che racchiude l'interfaccia utente (IOPort). Pur risiedendo sullo stesso hardware del `cargoservice`, la WebGUI mantiene un'identità logica separata in grado di comunicare via HTTP/WebSocket/CoAP.
- `ctx_picow`: Un contesto remoto (es. un microcontrollore Raspberry Pi Pico) in cui sono connessi fisicamente i dispositivi di I/O. All'interno di questo contesto posizioneremo le entità **sonar** e **led**, modellandole come attori capaci di ricevere e scambiare messaggi sulla rete con il `cargoservice`.

L'esito di questa fase è un'**architettura logica del sistema** definita ed eseguibile. Per l'analisi di dettaglio delle singole interazioni interne, la risoluzione delle problematiche legate allo stato del robot e la gestione dei timeout operativi, si rimanda all'implementazione formale dello **Sprint 1**.

Test plans

Coerentemente con la teoria dello Sprint 0, viene qui definito un primo **TestPlan di Functional Tests**. Questi test mirano a validare logicamente il comportamento del sistema desunto strettamente dai requisiti base, verificandone le macro-risposte (Happy Path e scenari di fallimento).

Test Funzionale 1: Carico Rifiutato (Stiva Piena)

Stato: Il `cargoservice` registra che gli slot da 1 a 4 sono occupati.

Azione: Invio richiesta `load`.

Risultato Atteso: Il sistema risponde immediatamente con il messaggio `reject` senza attivare la movimentazione robotica.

Riferimento Codice: Nel file `sprint0/src/cargoservice.qak`

(<https://github.com/riccardocar99/cargoservice-/blob/main/sprint0/src/cargoservice.qak>)

lo stato viene gestito in `handleLoadRequest` con la seguente guardia booleana:

```
} else if (Slot1Occupied && Slot2Occupied && Slot3Occupied && Slot4Occupied) {  
    replyTo load with reject : reject(hold_full)  
}
```

Test Funzionale 2: Accettazione Carico (Happy Path)

Stato: Stiva parzialmente vuota e sistema pienamente funzionante.

Azione: Invio richiesta `load`.

Risultato Atteso: Il sistema accetta la richiesta prenotando uno slot e fa lampeggiare il Led di segnalazione (sistema impegnato).

Organizzazione del processo di produzione

L'architettura logica QAK sviluppata (disponibile in [cargoservice.qak](https://github.com/riccardocar99/cargoservice-/blob/main/sprint0/src/cargoservice.qak) (<https://github.com/riccardocar99/cargoservice-/blob/main/sprint0/src/cargoservice.qak>)) e il corrispettivo TestPlan pongono le basi per l'avanzamento dei lavori. Di seguito la pianificazione strategica e operativa.

Definizione e Goal dello Sprint 1:

Goal dello Sprint 1: Realizzazione del nucleo centrale di orchestrazione (QActor `cargoservice`) e del driver di movimentazione del robot (QActor `cargorobot`), affrontando l'interazione reale con `RobotSmart26` (hostato in Docker) ed il superamento dell'abstraction gap cartesiano. Il prototipo eseguibile del ciclo di carico (Happy Path) deve essere convalidato tramite test automatici JUnit 4.

Motivazione:

La scelta di partire dal ciclo di carico principale (Happy Path) e dalla movimentazione del robot risponde alla necessità di mitigare il rischio tecnologico più elevato, ovvero l'integrazione di rete e fisica con il robot virtuale pre-fornito (`RobotSmart26`). Stabilire prima un'architettura logica di controllo funzionante e testabile programmaticamente consente di gettare le basi su cui innestare, nei successivi sprint, i dettagli tecnologici di contorno (quali la WebGUI reale e l'hardware del Sonar/Led su Raspberry Pi PicoW) riducendo al minimo l'incertezza nello sviluppo.

Testing

(Da compilare negli sprint successivi)

Deployment

(Da compilare al termine del progetto)

Maintenance

(N/A per Sprint 0)

Luigi



Riccardo



Simone



GIT repo: <https://github.com/riccardocar99/cargoservice->
(<https://github.com/riccardocar99/cargoservice->).